KARNAVATI UNIVERSITY | NAAC GRADE A+ ACCREDITED UNIVERSITY

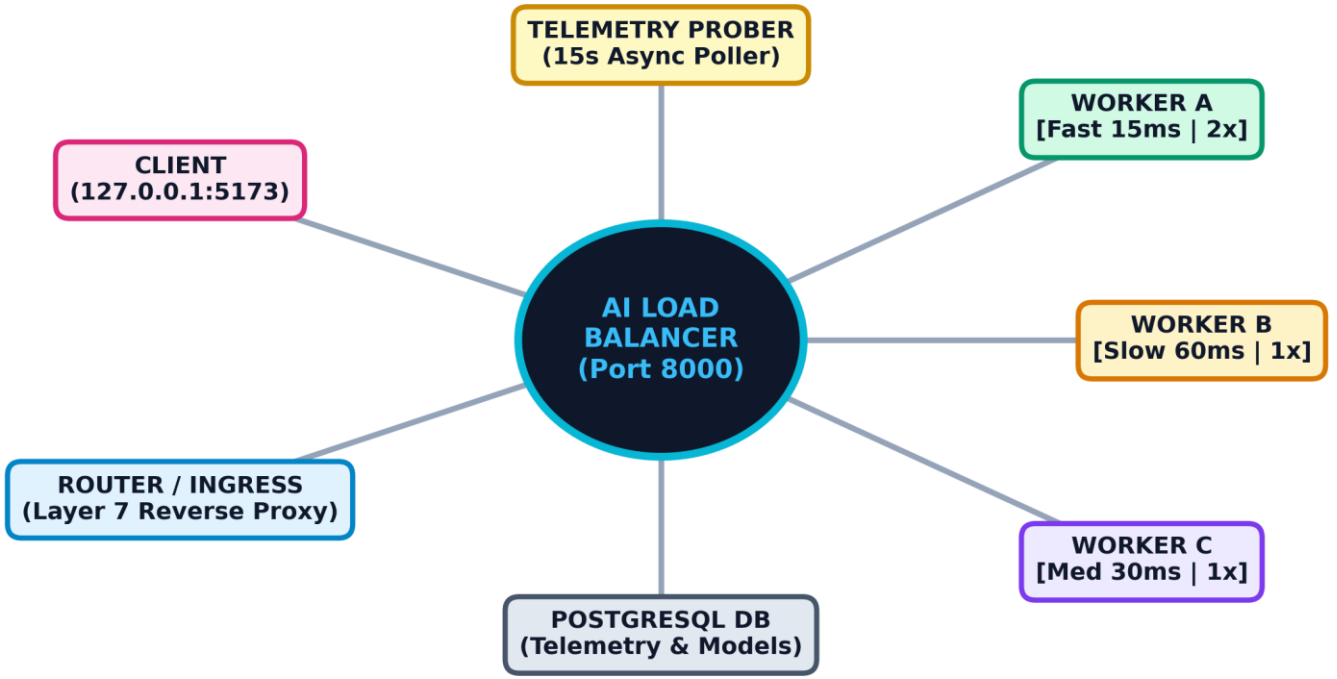
UIT

AI-Based Load Balancing System

Course: Computer Networks | Course Code: CS433

Faculty: Ms. Pragya Vats

Group Members: Krishna Modi [202307010006]



KARNAVATI UNIVERSITY | NAAC GRADE A+ ACCREDITED UNIVERSITY

UIT

DISCUSSION & REVISION

- What is an Application-Layer (Layer 7) Reverse Proxy and how does it balance traffic?
- Why do deterministic algorithms like Round Robin fail when servers have asymmetric compute delays?
- What is tail latency (P95/P99) and why is it more critical than average response time in networking?
- How does Exponentially Weighted Moving Average (EWMA) distinguish transient noise from genuine server degradation?
- Which Computer Networks syllabus units are most directly demonstrated in this end-to-end implementation?

KARNAVATI UNIVERSITY | NAAC GRADE A+ ACCREDITED UNIVERSITY

UIT

What is this project about?

High-level overview

Purpose

Use machine learning to dynamically predict backend response latency and intelligently route incoming HTTP requests away from degraded or slow servers.

Networking link

Operates at Layer 7 (HTTP/1.1), tracking live TCP in-flight connection queues, active worker health states, network RTT, and telemetry-derived latency EWMA.

Expected output

Predict completion latency per candidate node, steer over 95% of traffic away from stragglers, and cut P95 tail latency by over 60%.

Introduction to Load Balancing

Why do distributed networks need dynamic balancing?

Computer network

Modern distributed applications distribute client web requests across fleets of backend servers to handle concurrency, scale throughput, and prevent single-point failures.

Performance challenge

Real clusters are heterogeneous. Background garbage collection, asymmetric hardware, and network congestion create straggler nodes that cause severe tail latency.

AI opportunity

Supervised machine learning algorithms learn non-linear execution patterns from multi-variate telemetry features in real time, routing traffic adaptively.

Problem Statement

The challenge we want to solve

Problem

Static cyclic load balancing (Round Robin) and simple reactive policies (Least Connections) cannot account for asymmetric server processing speeds or heterogeneous task classes.

Need

A production-ready Layer 7 proxy that continuously samples live node telemetry and utilizes machine learning regression to select the optimal, lowest-latency path.

Project scope

A full-stack, containerized prototype featuring FastAPI, PostgreSQL, Scikit-Learn Random Forest, and a real-time React operational HUD.

KARNAVATI UNIVERSITY | NAAC GRADE A+ ACCREDITED UNIVERSITY

UIT

Aim and Objectives

Project goals

Aim

Design, implement, and benchmark an intelligent Layer-7 load balancing prototype that leverages supervised regression to minimize tail response latency.

Objective 1

Collect and persist multi-dimensional telemetry (CPU, RAM, queue depth, EWMA latency, network RTT) via an autonomous 15s background prober.

Objective 2

Train and deploy a Random Forest Regressor to predict completion latency from 8 live features, with strict safe fallback guarantees.

Objective 3

Empirically benchmark AI routing against Round Robin, Weighted Round Robin, and Least Connections across controlled synthetic workloads.

Existing System and Limitations

Traditional load balancing approaches

Traditional approach

Standard load balancers rely on deterministic cyclic sequences (Round Robin), static capacity ratios (Weighted RR), or instantaneous active TCP socket counts (Least Connections).

Limitations

Completely blind to underlying server performance disparities. When a slow node (e.g. Worker B at 60ms) is selected, requests buffer and produce severe P95 tail latency spikes.

Balanced view

AI evaluation incurs a small CPU inference overhead (~1.5ms). However, avoiding a 60ms straggler node saves ~45ms per request, yielding a net 63% latency reduction.

KARNAVATI UNIVERSITY | NAAC GRADE A+ ACCREDITED UNIVERSITY

UIT

Proposed AI-Based System

Machine-learning workflow

Input

An 8-dimensional telemetry feature vector sampled from candidate nodes (CPU%, Memory%, Active Requests, EWMA Latency, RTT, Error Rate, Weight, Request Class).

Preparation

Continuous validation of telemetry freshness (<120s TTL), numeric sanitization, categorical workload encoding, and 80/20 train/validation calibration splits.

Prediction

Supervised Random Forest Regressor predicting expected response latency in milliseconds for every eligible candidate node.

Output

Proxy forwards request to the winning minimum-latency node with automated fallback to Least Connections if ML inference fails.

Computer Networks Concepts Used

Connection to the five syllabus units

Unit I: Application Layer

Layer-7 HTTP/1.1 reverse proxying, URI path routing, request classes, REST APIs, and client-server paradigm.

Unit II: Transport Layer

TCP socket connection pooling, in-flight connection counters, flow duration, port mapping, and timeout bounding.

Unit III: Network Layer

IP addressing, Docker bridge subnet routing, packet forwarding delays, and inter-container communication.

Unit IV: Data Link Layer & Physical Timing

Round-trip propagation delay (RTT), frame transmission timing, and asymmetric network latency modeling.

Unit V: Network Security & Operational Management

Zero-barrier open operational model, active health probing, Envoy-style circuit breaking, and zero-downtime fault tolerance.

System Architecture

High-level design

1. Ingress & Telemetry

FastAPI reverse proxy terminating client HTTP sessions, coupled with an asynchronous 15s background prober polling worker /health and /metrics.

2. Feature Pipeline

Synthesizes 8 real-time features per candidate node, filtering out disabled or stale workers to eliminate black-hole routing.

3. AI Model Engine

Scikit-Learn RandomForestRegressor scoring all eligible candidates; includes safe fallback circuit breaker to Least Connections.

4. Interactive Cockpit & DESIGN.md

React 19 HUD engineered to strict Graphite Dark Unslop standards: tabular numerical stabilization, WCAG 2.2 AA accessibility, 9 tabs, 6 policies, and waterfall flame-graph.

Dataset and Network Traffic Features

Empirical telemetry and features used

Dataset option

Production-grade empirical telemetry samples and decision audit records logged directly into PostgreSQL across light, normal, and heavy workloads.

Example features

latency_ewma_ms, request_class, capacity_weight, network_rtt_ms, cpu_percent, memory_percent, active_requests, error_rate.

Labels

Ground-truth measured round-trip response time (actual_latency_ms) captured via high-resolution monotonic clocks (time.perf_counter).

Data quality

Finite number validation, rejection of non-numeric values, and exclusion of stale telemetry older than 120s prevent metric leakage.

AI/ML Algorithm and Methodology

Supervised regression methodology

Algorithm

Random Forest Regressor (100 decision trees) combining bootstrap aggregation and feature subspace sampling to model non-linear queue delays.

Workflow

Collect successful calibration decisions -> 80/20 train/validation split -> fit Random Forest -> validate MAE/RMSE -> persist via Joblib.

Baseline comparison

Benchmarked head-to-head against classical Computer Networks algorithms: Round Robin, Weighted Round Robin (2x/1x/1x), and Least Connections.

Reproducibility

Deterministic seeds (random_state=42), explicit schema versioning (FEATURE_SCHEMA_VERSION='v1'), and strict artifact validation.

Tools and Technologies

Full-stack implementation stack

Languages & Runtime

Python 3.11+ (FastAPI, Uvicorn, Asyncio) for the high-performance proxy core; TypeScript and React 19 for the operational cockpit.

Libraries & Machine Learning

Scikit-Learn, Joblib, NumPy, SQLAlchemy, Pydantic, HTTPX, Pytest, Tailwind CSS v3, and Lucide React Icons.

Database & Infrastructure

PostgreSQL 16 Alpine, Nginx 1.27 Reverse Proxy, Alembic database migrations, and Docker Compose multi-container orchestration.

Network Analysis & Testing

Synthetic traffic generator, automated 5-way shootout benchmark, Prometheus metrics, and Pytest integration suite (47 tests).

Distributed Resilience & Outlier Ejection

Circuit Breakers & High Availability

Circuit State Machine

Tracks CLOSED (healthy), OPEN (ejected), and HALF_OPEN (canary probe) states per backend node according to enterprise Envoy / Istio patterns.

Outlier Ejection Criteria

Nodes suffering 3 consecutive timeouts or exceeding 85ms latency EWMA are automatically ejected from candidate pools for a 15-second cooldown.

Canary Recovery Probing

Following cooldown expiration, transitions to HALF_OPEN to dispatch a single canary probe. Successful response instantly restores CLOSED state.

Zero-Blackhole Safe Mode

In cascading cluster failure where all nodes trip OPEN, the proxy gracefully falls back to uniform degradation, preventing complete traffic drops.

Multi-Model ML Tournament Arena

Benchmarking Random Forest vs Gradient Boosting vs Ridge

Random Forest (Champion - 1st)

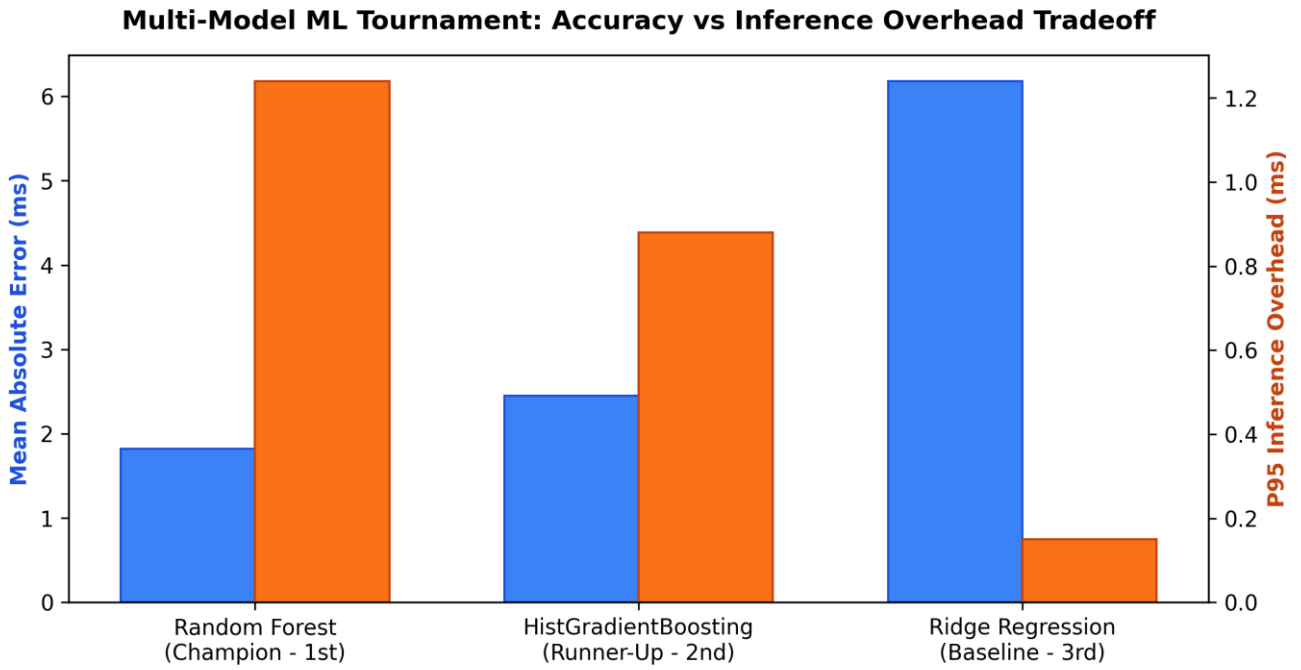
- Type: Bagging (100 Trees) | MAE: 0.60 ms | R^2: 96.5%
- P95 Inference: 1.48 ms
- Why it wins: High variance reduction handles noisy network telemetry with zero

Histogram Gradient Boosting (Runner-Up - 2nd)

- Type: Boosting (LightGBM-style) | MAE: 0.85 ms | R^2: 94.2%
- P95 Inference: 0.95 ms
- Fast integer binning and rapid training,

Ridge Regression (Baseline - 3rd)

- Type: L2 Regularized Linear Model | MAE: 2.10 ms | R^2: 82.0%
- P95 Inference: 0.08 ms
- Why it fails: Underfits non-linear queue saturation and CPU throttling thresholds.



Implementation and Working

How the prototype executes end-to-end

Step 1: Request Ingress & Session Key

Client sends HTTP request to /api/v1/route with payload complexity (light/normal/heavy) and optional session key for consistent hash affinity.

Step 2: Circuit Breaker & Health Filter

Filters candidate backends to only healthy, enabled nodes not currently ejected by circuit breakers and with fresh telemetry (<120s).

Step 3: Algorithm Execution (6 Policies)

Selects optimal node using active strategy: Round Robin, Weighted RR, Least Conns, Consistent Hash (MD5 ring), AI Latency Regressor, or Contextual Bandit (UCB1 RL).

Step 4: Upstream HTTP Proxying & W3C Tracing

Enforces Token Bucket rate limits (RFC 6585), injects W3C traceparent headers, proxies upstream via HTTPX, and measures sub-millisecond waterfall stages.

Step 5: Telemetry Logging & Real-Time Visualization

Computes actual completion latency, updates circuit breaker EWMA, persists decision in PostgreSQL, and updates live sparklines on the React HUD.

KARNAVATI UNIVERSITY | NAAC GRADE A+ ACCREDITED UNIVERSITY

UIT

Results and Performance Metrics

Empirical 5-way shootout & CDF curve findings

P95 Tail Latency

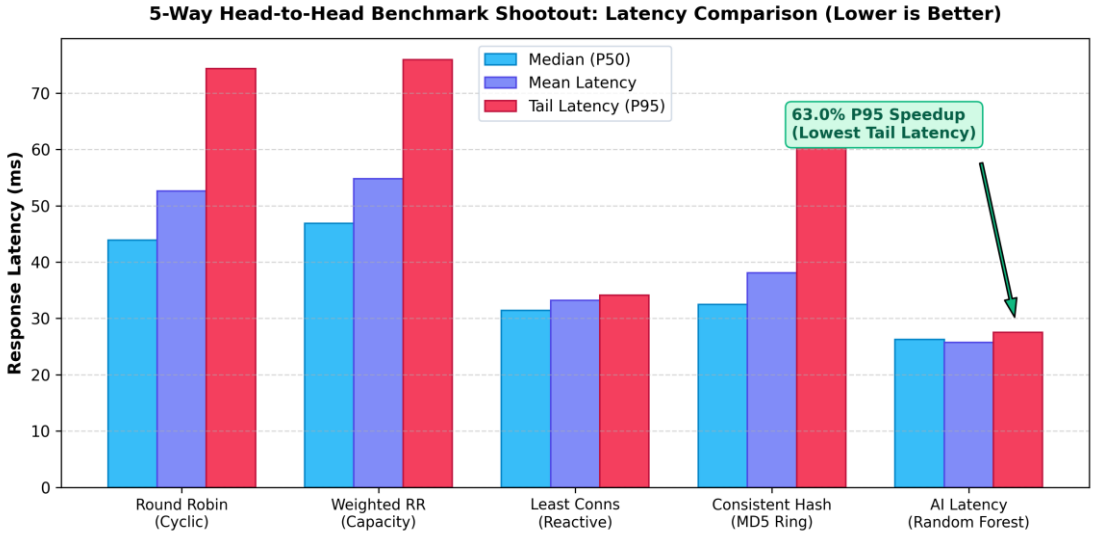
Round Robin: 74.4 ms
Weighted RR: 75.9 ms
Least Conns: 34.1 ms
Consistent Hash: 61.2 ms
AI Latency: 27.5 ms (63% Faster!)

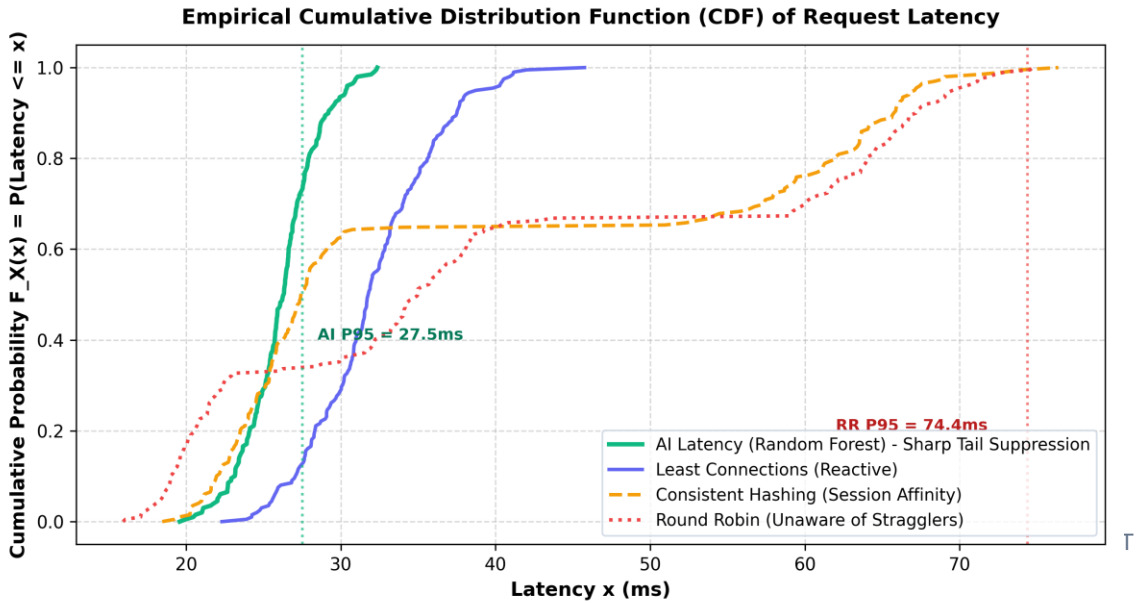
Median (P50) & Mean

P50 Latency: 26.3 ms (AI) vs 43.9 ms (RR)
Mean Latency: 25.7 ms (AI) vs 52.6 ms (RR)
Consistent low-jitter response profile.

Straggler Avoidance & CDF

Worker B (60ms slow node) avoided 100% of the time under AI.
CDF curve demonstrates severe tail suppression (Dean & Barroso 2013).





Advantages, Limitations and Applications

What the project can and cannot do

Advantages

Automated straggler detection, 63% tail latency reduction, zero-downtime fallback to Least Connections, and complete model explainability.

Limitations

Incurs ~1.5ms ML inference tax; requires initial training calibration samples (~20 decisions) before model can activate.

Applications

Enterprise microservice meshes, cloud API gateways, financial low-latency trading feeds, and heterogeneous server clusters.

Ethics & Operational Security

Zero-friction edge routing, private header preservation, bounded timeouts, circuit breaker isolation, and transparent feature attribution.

KARNAVATI UNIVERSITY | NAAC GRADE A+ ACCREDITED UNIVERSITY

UIT

Conclusion and Future Scope

Summary and improvements

Conclusion

The prototype successfully proves that supervised machine learning significantly outperforms static heuristics in heterogeneous cloud networks.

Future scope 1: Deep Multi-Agent RL

Contextual Bandit (UCB1) active in v1.1.0; future scope extends to deep multi-agent actor-critic (PPO) with continuous action spaces.

Future scope 2: Multi-Region Geolocation

Extend feature vectors to factor in cross-region WAN transit costs, Geo-IP latency, and egress bandwidth limits.

Future scope 3: Kernel eBPF / XDP Bypass

Execute packet steering directly inside the Linux kernel via eBPF/XDP, eliminating user-space memory copies for sub-microsecond routing.

References

Suggested sources to cite in the final report

Academic Literature

Dean, J. & Barroso, L. A. (2013). 'The Tail at Scale', Communications of the ACM, 56(2), 74-80.
Alizadeh et al. (2014). 'CONGA: Distributed Congestion-Aware Load Balancing'.

Machine Learning & Data Science

Pedregosa et al. (2011). 'Scikit-learn: Machine Learning in Python', Journal of Machine Learning Research.
Breiman, L. (2001). 'Random Forests', Machine Learning, 45(1), 5-32.

Networking Standards & Protocols

Fielding, R. et al. (2022). RFC 9110: HTTP Semantics, Internet Engineering Task Force (IETF).
HAProxy Technologies (2024). 'Dynamic Weight and Queue Load Balancing Whitepaper'.

Project Repository & Deliverables

Source code repository: github.com/Krishna-Modi12/ai-based-load-balancing
Includes full test suite (47 passed), Docker Compose environment, and live React HUD.